
SynthSAEBench: Evaluating Sparse Autoencoders on Scalable Realistic Synthetic Data

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Improving Sparse Autoencoders (SAEs) requires benchmarks that can precisely val-
2 idate architectural innovations. Current LLM-based SAE benchmarks are too noisy
3 to differentiate architectural improvements, while commonly used synthetic-data
4 experiments are too small-scale, unstandardized, and unrealistic to be meaningful.
5 We introduce **SynthSAEBench**, a benchmark and toolkit for evaluating SAEs
6 against large-scale synthetic data with realistic feature characteristics including
7 correlation, hierarchy, and superposition, while providing ground-truth features
8 and firings. SynthSAEBench acts as a controlled lower-bound test: SAE architec-
9 tures that fail when the Linear Representation Hypothesis holds by construction
10 have little hope on real LLMs. The benchmark reproduces known LLM SAE phe-
11 nomena including the disconnect between reconstruction and latent quality, poor
12 SAE probing, and a precision-recall trade-off mediated by L0, demonstrating that
13 SynthSAEBench findings reproduce results on LLM SAEs. We further identify a
14 novel failure mode: Matching Pursuit SAEs exploit superposition noise to improve
15 reconstruction without learning ground-truth features, suggesting more expres-
16 sive encoding procedures can easily overfit. SynthSAEBench complements LLM
17 benchmarks with ground-truth features and controlled ablations for diagnosing
18 SAE failure modes, while providing a clear target for SAE architecture work.

19 1 Introduction

20 Large language models (LLMs) achieve remarkable performance but remain opaque, motivating
21 interpretability research into how these models represent knowledge. The Linear Representation
22 Hypothesis (LRH) [29] posits that concepts (hereafter “features”) are represented as nearly-orthogonal
23 linear directions. Models can represent many more features than dimensions by allowing non-
24 orthogonal directions, a phenomenon known as superposition [15]. Superposition is efficient but
25 makes interpreting activations difficult, motivating the use of Sparse Autoencoders (SAEs) [5, 14] to
26 recover underlying feature directions via sparse dictionary learning.

27 A key challenge in improving SAEs is that we lack ground-truth knowledge of the “true features” in
28 an LLM. LLM benchmarks such as SAEbench [24] evaluate SAE performance on tasks like sparse
29 probing [21, 22], concept disentanglement [23], and autointerpretability [30]. However, SAEbench
30 metrics exhibit substantial noise between runs (see Appendix J), making it difficult to evaluate small
31 architectural improvements. Moreover, without ground-truth access, we cannot diagnose *why* SAEs
32 score poorly, a critical obstacle given recent work showing that SAEs underperform supervised
33 methods like logistic-regression probes [22].

34 On the other extreme, SAE research already relies on bespoke synthetic models: toy experiments with
35 fewer than 10 independent features [33, 19, 15], or per-paper setups studying hierarchy [11, 13, 7]

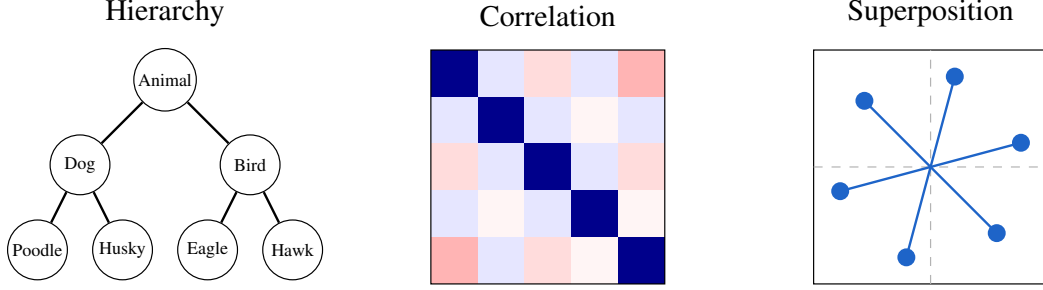


Figure 1: SynthSAEBench provides a large-scale synthetic data model with realistic feature characteristics including correlation, hierarchy, superposition and zipfian firing distributions, scalable to hundreds of thousands of features and realistic hidden dimension sizes.

and correlation [10, 9]. Nearly all SAE architecture work already involves experiments with synthetic models, but these models are too small, unrealistic, and unstandardized to compare architectures.

We introduce **SynthSAEBench**, an extension of the SAE lens library [4] that scales the synthetic model approach to 16k+ features with correlation, hierarchy, and superposition, providing ground-truth feature directions and firings. Data generation runs at hundreds of thousands of samples per second on a single GPU. We release a standard pretrained configuration as the canonical benchmark, while allowing practitioners to change any part of the synthetic model to perform ablations and test SAE performance under different feature assumptions.

SynthSAEBench is a controlled lower-bound test for SAE architectures: If SAEs fail when the Linear Representation Hypothesis holds by construction – the setting SAEs are explicitly designed around – then we cannot expect them to succeed on real LLMs. Indeed, most previously observed LLM SAE phenomena also emerge in this regime: (1) Matryoshka SAEs overperform on SAE Bench despite poor reconstruction [7, 24], (2) Matching Pursuit SAEs overperform on reconstruction while scoring poorly on SAE Bench [8], (3) poor SAE probing performance [22], and (4) a precision-recall trade-off in probing mediated by SAE L0 [11], and (5) BatchTopK and JumpReLU both being nearly equivalent state-of-the-art architectures [4, 6, 31]. This shows that performance on SynthSAEBench transfers to results on real LLM SAEs.

Additionally, using SynthSAEBench, we find that Matching Pursuit SAEs exploit superposition noise to improve reconstruction without learning ground-truth features, hinting at why the simple linear encoder of traditional SAEs is so hard to outperform despite known theoretical limitations [28].

No SAE architecture we evaluate achieves perfect performance on SynthSAEBench, highlighting a clear target for improvement in SAE architectures.

Code is available on Anonymous Github¹.

2 Background

Sparse autoencoders (SAEs). An SAE decomposes an input activation $a \in \mathbb{R}^D$ into a hidden state f consisting of L hidden neurons, called “latents”. An SAE is composed of an encoder $W_{\text{enc}} \in \mathbb{R}^{L \times D}$, a decoder $W_{\text{dec}} \in \mathbb{R}^{D \times L}$, a decoder bias $b_{\text{dec}} \in \mathbb{R}^D$, and an encoder bias $b_{\text{enc}} \in \mathbb{R}^L$, and a nonlinearity σ , typically ReLU or a variant like JumpReLU [31], TopK [18] or BatchTopK (BTK) [6].

$$f = \sigma(W_{\text{enc}}(a - b_{\text{dec}}) + b_{\text{enc}}) \quad (1)$$

$$\hat{a} = W_{\text{dec}}f + b_{\text{dec}} \quad (2)$$

The SAE is trained with a reconstruction loss, typically Mean Squared Error (MSE), and a sparsity-inducing loss consisting of a function $\mathcal{S}$ that penalizes non-sparse representation with corresponding sparsity coefficient λ . For standard L1 SAEs, $\mathcal{S}$ is the L1 norm of f . For TopK and BatchTopK SAEs,

¹Code: <https://anonymous.4open.science/r/synth-sae-bench-anon-0C61>

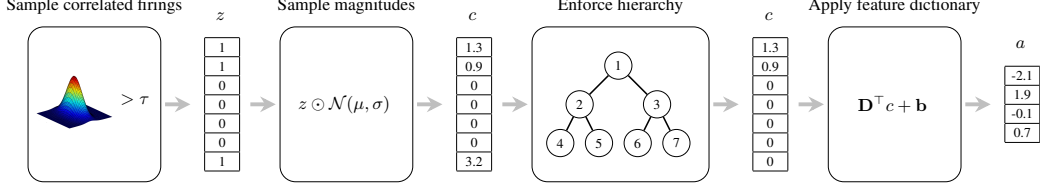


Figure 2: Overview of process to generate a single training activation, a .

there is no sparsity-inducing loss ($\mathcal{S} = 0$) as the TopK function directly induces sparsity. There is sometimes also an additional auxiliary loss $\mathcal{L}_{aux}$ with coefficient α to ensure all latents fire. Standard L1 SAEs typically do not have an auxiliary loss [27]. The general SAE loss is

$$\mathcal{L} = \|a - \hat{a}\|_2^2 + \lambda \mathcal{S} + \alpha \mathcal{L}_{aux}. \quad (3)$$

Matryoshka SAEs. A matryoshka SAE [7] extends the SAE definition by summing losses created by prefixes of SAE latents. This forces each sub-SAE to reconstruct input activations on its own, and incentivizes the SAE to place more common, general concepts into latents with smaller index number. A matryoshka SAE uses nested prefixes with sizes $\mathcal{M} = m_1, m_2, \dots, m_n$ where $m_1 < m_2 < \dots < m_n = L$, where L is the number of latents in the full dictionary. Matryoshka SAE loss is:

$$\mathcal{L} = \sum_{m \in \mathcal{M}} (\|a - \hat{a}_m\|_2^2 + \lambda \mathcal{S}_m) + \alpha \mathcal{L}_{aux} \quad (4)$$

Where $\hat{a}_m$ is the reconstruction for the SAE using the first m latents, and $\mathcal{S}_m$ is the sparsity penalty applied to the first m latents. For TopK and BatchTopK Matryoshka SAEs, there is no sparsity penalty ($\mathcal{S}_m = 0$) as the TopK function directly imposes sparsity.

Matching Pursuit SAEs. A Matching Pursuit (MP) SAE [13] acts like a TopK SAE where the k latents are selected in serial rather than in parallel. In an MP-SAE, there is no explicit encoder W_{enc} or encoder bias b_{enc} . Instead, at each iteration $t \in \{0, \dots, k-1\}$, the latent with the highest projection onto the residual r_t is selected and its contribution is projected out:

$$l_t = \operatorname{argmax}_i W_{dec,i}^\top r_t \quad (5)$$

$$r_{t+1} = r_t - (W_{dec,l_t}^\top r_t) W_{dec,l_t} \quad (6)$$

with $r_0 = a$. The SAE latent vector f accumulates the projection coefficients across iterations into the corresponding latent indices:

$$f_l = \sum_{t: l_t=l} W_{dec,l_t}^\top r_t, \quad (7)$$

so a latent selected on multiple iterations has its activations summed. The MP-SAE is trained with the standard MSE reconstruction loss $\|a - \hat{a}\|_2^2$, where the MP-SAE reconstruction is $\hat{a} = W_{dec} f$ (no decoder bias); by construction, this equals $\|r_k\|_2^2$.

The variant of MP-SAEs we use in this paper does not do any early stopping based on $\|r\|_2$ or based on selecting the same latent multiple times, as early stopping adds more complication to the training process and has not been shown to improve results [8].

3 Synthetic Data Model

We now describe our synthetic data model in detail. We extend the traditional bernoulli-gaussian model commonly used in dictionary learning literature [19, 35] and follows the LRH by construction. This is also the standard synthetic setting used by SAE research [33, 28, 7].

Our model contains a feature dictionary $\mathbf{D} \in \mathbb{R}^{N \times D}$ containing N ground-truth feature directions, each represented as a unit-norm vector $\mathbf{d}_i \in \mathbb{R}^D$, an optional bias $\mathbf{b} \in \mathbb{R}^D$, and an activation generator that samples sparse feature coefficients. Each feature d_i has a corresponding firing probability p_i .

Given a sample, the activation generator first determines which features fire using a Gaussian copula approach to generate correlated binary firing indicators. First, we sample from a multivariate Gaussian with correlation structure Σ :

$$\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \Sigma). \quad (8)$$

We then threshold these samples to obtain binary firing indicators that respect both the marginal firing probabilities p_i and the correlation structure. For feature i with firing probability p_i , we compute the threshold as $\tau_i = \Phi^{-1}(1 - p_i)$, where Φ^{-1} is the inverse standard normal CDF, and set $z_i = \mathbf{1}[g_i > \tau_i]$. When $\Sigma = \mathbf{I}$ (no correlations between features), this is equivalent to $z_i \sim \text{Bernoulli}(p_i)$.

For features that fire ($z_i = 1$), coefficients are sampled from a rectified Gaussian distribution:

$$c_i = z_i \cdot \text{ReLU}(\mu_i + \sigma_i \epsilon_i), \quad \epsilon_i \sim \mathcal{N}(0, 1), \quad (9)$$

where μ_i and σ_i are the per-feature mean and standard deviation of firing magnitudes. Optionally, a post-processing function $h : \mathbb{R}^N \rightarrow \mathbb{R}^N$ can be applied to modify the coefficient vector, i.e., $\mathbf{c} \leftarrow h(\mathbf{c})$; we use this mechanism to implement hierarchical constraints (§3.3).

The hidden activation is then computed as:

$$\mathbf{a} = \sum_{i=1}^N c_i \mathbf{d}_i + \mathbf{b} = \mathbf{D}^\top \mathbf{c} + \mathbf{b}. \quad (10)$$

For scalability with large numbers of features, we use a low-rank correlation matrix:

$$\Sigma = \mathbf{F}\mathbf{F}^\top + \text{diag}(\boldsymbol{\delta}), \quad (11)$$

where $\mathbf{F} \in \mathbb{R}^{N \times r}$ is a factor matrix of rank $r \ll N$, and $\boldsymbol{\delta} \in \mathbb{R}^N$ contains diagonal residual variances chosen to ensure unit diagonal in Σ (i.e., $\delta_i = 1 - \sum_j F_{ij}^2$). Sampling from this structure is efficient:

$$\mathbf{g} = \mathbf{F}\boldsymbol{\epsilon} + \sqrt{\boldsymbol{\delta}} \odot \boldsymbol{\eta}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_r), \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N), \quad (12)$$

where $\odot$ denotes elementwise multiplication. This requires only $O(Nr)$ computation per sample rather than $O(N^2)$ for full covariance sampling.

This generative model enables us to control several important phenomena that arise in real neural networks: superposition (§3.1), feature correlation (§3.2), and feature hierarchy (§3.3). Both per-feature firing probabilities p_i and magnitude parameters (μ_i, σ_i) admit configurable distributions; we describe the supported options in Appendix E. Figure 2 shows the data generating process.

3.1 Superposition

The Linear Representation Hypothesis [29] posits that neural networks represent more concepts than they have dimensions, forcing features to share representational capacity—a phenomenon known as superposition [15]. We characterize the degree of superposition by *mean max absolute cosine similarity*, ρ_{mm} . $\rho_{\text{mm}} = \frac{1}{N} \sum_{i=1}^N \max_{j \neq i} |\mathbf{d}_i^\top \mathbf{d}_j|$. $0 \leq \rho_{\text{mm}} \leq 1$, with $\rho_{\text{mm}} = 0$ indicating no superposition (i.e., all features are mutually orthogonal).

Feature vectors are initialized as random unit vectors sampled from a standard normal distribution:

$$\mathbf{d}_i \leftarrow \frac{\mathbf{g}_i}{\|\mathbf{g}_i\|_2}, \quad \mathbf{g}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D). \quad (13)$$

While this initialization produces features with small expected pairwise cosine similarities (scaling as $O(1/\sqrt{D})$), some feature pairs may have higher overlap by chance. To reduce spurious correlations between feature directions, we optionally apply an orthogonalization procedure to minimize pairwise cosine similarity. Specifically, we optimize:

$$\mathcal{L}_{\text{ortho}} = \sum_{i \neq j} (\mathbf{d}_i^\top \mathbf{d}_j)^2 + \lambda \sum_i (\|\mathbf{d}_i\|_2 - 1)^2 \quad (14)$$

130 using gradient descent. After orthogonalization, all vectors are renormalized to unit length. This
 131 procedure encourages feature vectors to be as orthogonal as possible given the dimensionality
 132 constraints, although when using models with a large hidden dimension, we find that random
 133 initialization already results in features that are nearly orthogonal.

134 For scalability with large N , we use a memory-efficient chunked implementation that computes
 135 pairwise dot products in blocks. This reduces memory complexity from $O(N^2)$ to $O(\text{chunk_size} \times$
 136 $N)$, enabling orthogonalization of dictionaries with thousands or even millions of features.

137 3.2 Feature Correlation

138 Real neural network features are rarely independent: concepts that co-occur in data tend to co-fire in
 139 the network’s representations. Previous work has shown that correlated features present challenges
 140 for SAEs, leading to phenomena like feature hedging [10, 9]. To study these effects systematically,
 141 we implement configurable correlation structures between feature firings.

142 We support randomly generating a low-rank correlation matrix for use in the synthetic model. This
 143 random generation is controlled by two parameters: the rank r (which determines the complexity of
 144 the correlation patterns) and a correlation scale s (which controls the magnitude of off-diagonal corre-
 145 lations by scaling the factor matrix). The detailed generation procedure is described in Appendix D.

146 3.3 Feature Hierarchy

147 Concepts in natural language and vision exhibit hierarchical structure: “golden retriever” is a type of
 148 “dog,” which is a type of “animal.” Previous work has shown that SAEs struggle with hierarchical
 149 features, leading to phenomena like feature absorption [11]. We implement configurable hierarchical
 150 dependencies between features to study these effects.

151 Our hierarchy is represented as a forest of trees, where each node corresponds to a feature and
 152 children can only fire when their parent is active. Formally, after sampling the initial firing indicators
 153 z_i , we apply the constraint:

$$c_{\text{child}} \leftarrow c_{\text{child}} \cdot \mathbf{1}[c_{\text{parent}} > 0], \quad (15)$$

154 which zeros out child activations whenever the parent is inactive.

155 Additionally, we support *mutual exclusion* among siblings: when a parent node is marked as having
 156 mutually exclusive children, at most one child can be active per sample. This models concepts like
 157 “dog” vs. “bird”—both are animals, but a single entity cannot be both simultaneously. When multiple
 158 siblings would fire, one is randomly selected as the winner and the others are deactivated.

159 We also support *parent-scaled magnitudes*, where child activation magnitudes are modulated by their
 160 parent’s activation strength rather than simply being binary-gated. This models the intuition that the
 161 intensity of a specific concept (e.g., “golden retriever”) should scale with the intensity of its parent
 162 concept (e.g., “dog”). Details are in Appendix C.

163 For efficiency, we precompute the hierarchy structure as sparse index tensors, enabling
 164 $O(\text{active features})$ processing rather than $O(N)$ per sample.

165 4 Evaluating SAEs on Synthetic Data

166 A key advantage of synthetic data is access to ground-truth feature directions and activations, enabling
 167 precise evaluation of SAE quality. We implement a comprehensive set of metrics organized into four
 168 categories: reconstruction quality, feature recovery, classification accuracy, and sparsity.

169 4.1 Reconstruction Metrics

170 **Explained Variance (R^2).** We measure how well the SAE reconstruction $\hat{\mathbf{a}}$ captures the variance
 171 in the input activations $\mathbf{a}$:

$$R^2 = 1 - \frac{\mathbb{E}[\|\mathbf{a} - \hat{\mathbf{a}}\|_2^2]}{\text{Var}(\mathbf{a})}, \quad (16)$$

172 where $\text{Var}(\mathbf{a}) = \mathbb{E}[\|\mathbf{a}\|_2^2] - \|\mathbb{E}[\mathbf{a}]\|_2^2$ is the total variance. A value of 1.0 is perfect reconstruction.

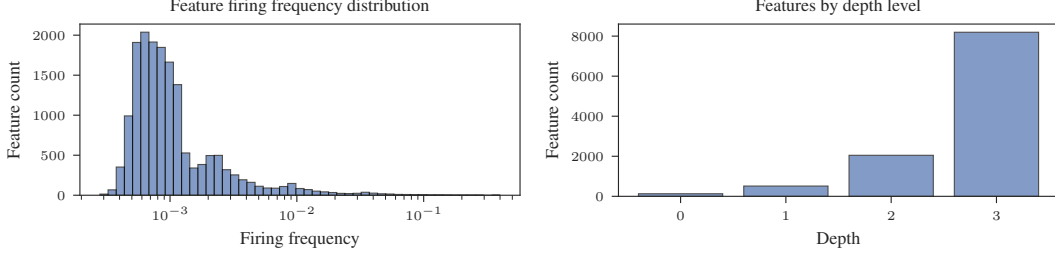


Figure 3: SynthSAEBench feature firing probabilities (left) and hierarchy distribution (right).

4.2 Feature Recovery Metrics

Mean Correlation Coefficient (MCC). Following Song et al. [33], we measure how well SAE decoder columns $\mathbf{w}_j$ align with ground-truth feature directions $\mathbf{d}_i$ using optimal bipartite matching. We compute the absolute cosine similarity matrix $|S_{ij}| = |\mathbf{w}_j^\top \mathbf{d}_i|$ (both $\mathbf{w}_j$ and $\mathbf{d}_i$ are unit-norm), find the optimal one-to-one matching via the Hungarian algorithm, and report the mean similarity of matched pairs:

$$\text{MCC} = \frac{1}{\min(L, N)} \sum_{(i,j) \in \text{matching}} |\mathbf{w}_j^\top \mathbf{d}_i|. \quad (17)$$

Feature Uniqueness. We measure what fraction of SAE latents track unique ground-truth features. For each latent j , we find its best-matching ground-truth feature: $i^*(j) = \arg \max_i |\mathbf{w}_j^\top \mathbf{d}_i|$. Uniqueness is the fraction of unique best matches divided by the number of latents:

$$\text{Uniqueness} = \frac{|\{i^*(j) : j = 1, \dots, L\}|}{L}. \quad (18)$$

A value of 1.0 means each latent tracks a different feature.

4.3 Classification Metrics

We evaluate each SAE latent as a binary classifier for its best-matching ground-truth feature. For latent j matched to feature $i^*(j)$, we compute standard classification metrics over evaluation samples: precision, recall, and F1 score. A latent is considered to fire when $f_j > 0$, and a ground-truth feature is considered to fire when $z_i = 1$. We report the mean of each metric across all latents.

4.4 Sparsity Metrics

L0 Comparison. We track the average L0 (number of active features per sample) of both ground-truth activations and for SAE latent activations.

Dead Latents. We count the number of SAE latents that never fire across the evaluation set. Dead latents represent wasted capacity and indicate training issues.

5 Benchmark Configuration

We define the standard SynthSAEBench configuration, chosen to be large-scale enough to elicit realistic SAE training dynamics while remaining tractable to run.

The model has $N = 16,384$ features in hidden dimension $D = 768$, yielding $\rho_{\text{mm}} \approx 0.15$. We estimate this is similar to the level of superposition 1B features would cause in a Gemma-2-2b sized model (Appendix K). Activations have mean L2 norm 28 and stdev 5, matching Pythia-160m [3] layer 10. Feature vectors are random unit vectors orthogonalized for 100 steps ($\text{lr } 3 \times 10^{-4}$); the bias is a random Gaussian direction normalized to magnitude 0.5. Base firing probabilities follow a Zipfian distribution ($\alpha = 0.5$, $p_{\text{max}} = 0.4$, $p_{\text{min}} = 5 \times 10^{-4}$); firing magnitudes have means interpolating linearly from 5.0 to 4.0, with stdevs $\sigma_i \sim |\mathcal{N}(0.5, 0.5^2)|$.

203 The hierarchy has 128 root trees with branching factor 4 and max depth 3, covering 10,880 features.
 204 All sibling nodes are mutually exclusive, child magnitudes scale with parent activation (Appendix C),
 205 and base probabilities are compensated for hierarchy and mutual exclusion effects (Appendix B).
 206 Beyond the hierarchy, a random low-rank correlation with rank $r = 25$ and scale $s = 0.1$ adds
 207 structured dependencies. The model has an L0 of 34, determined by the firing-probability distribution.

208 5.1 Benchmark instructions

209 We recommend training SAEs of width 4096 on SynthSAEBench, intentionally more narrow than the
 210 model dictionary, since LLM SAEs are always narrower than the (unknown) number of true features
 211 in an LLM [34]. Unless otherwise specified, we train SAEs on 200M samples (15–20 minutes per
 212 SAE on a single H100, batch size 1024). Sampling performance is in Appendix I.

213 6 Results

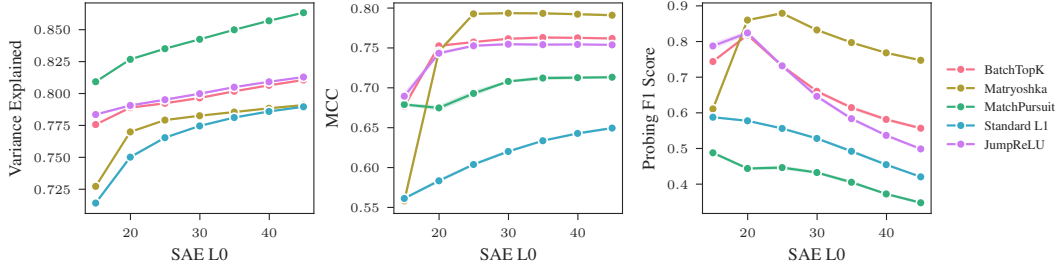


Figure 4: Variance explained (left), MCC (middle), and F1-score (right) for SynthSAEBench SAEs with varying L0. Shaded area is stdev with 5 seeds (too small to be visible for most SAEs).

214 We train standard L1, Matching Pursuit, BatchTopK, Matryoshka BatchTopK, and JumpReLU
 215 SAEs on SynthSAEBench. These SAEs have width 4096 and are trained on 200M samples, as
 216 recommended in Section 5.1. For standard L1 SAEs and JumpReLU SAEs, we implement a
 217 controller to automatically tune their sparsity coefficient during training to hit a target L0. This
 218 controller is described in more detail in Appendix G.

219 We vary the L0 of the SAEs from 15 to 45 with 5 seeds per L0. F1-score, MCC, and variance
 220 explained are shown in Figure 4. Matryoshka SAEs achieve the best probing and MCC scores,
 221 indicating the best latent quality, despite poor explained variance (poor reconstruction). This matches
 222 the results of Matryoshka SAEs on SAEBench [23], where this same pattern is observed for LLM
 223 SAEs. The performance drop-off for Matryoshka SAEs at low L0 seems to be due to dead latents
 224 rather than a fundamental architectural issue (see Appendix H). MP-SAEs, on the other hand, have the
 225 best variance explained, but have poor probing and MCC results, indicating that MP-SAEs impressive
 226 reconstruction comes at the cost of latent quality. We see JumpReLU and BatchTopK achieving
 227 roughly similar levels of quality and reconstruction, further validating why these are both considered
 228 state-of-the-art architectures.

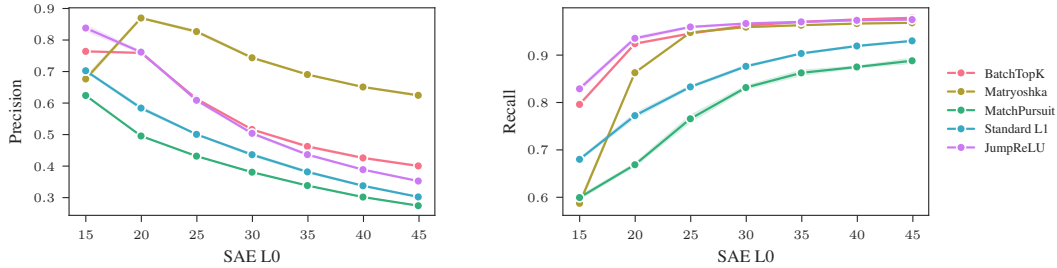


Figure 5: Probing precision and recall for SAEs trained on SynthSAEBench across varying L0 values. Higher L0 increases recall at the cost of precision. Shaded area is stdev.

6.1 Precision-recall trade-off mediated by L0

One striking result from Figure 4 is that no SAE achieves near-perfect probing F1 at any L0, with the best performer, Matryoshka SAEs, peaking at around F1=0.88. This means no SAEs will act as a great classifier for ground-truth model features. This is consistent with results showing that LLM SAEs underperform logistic-regression probes [22]. Indeed, logistic regression probes trained directly on SynthSAEBench activations achieve a mean F1 of 0.974 (Appendix L), substantially outperforming the best SAE. This poor probing performance is one of the key criticisms of SAEs, so our ability to reproduce it in a setting with known ground-truth features is a key first step to address this problem with SAE architectural improvements.

We show the precision and recall for the probing task in Figure 5. Higher L0 increases recall at the cost of precision, reproducing the precision-recall trade-off mediated by SAE L0 seen in previous LLM SAE studies [24, 11].

6.2 Near-equivalence between JumpReLU and BatchTopK

JumpReLU and BatchTopK are both considered state-of-the-art architectures [4, 31, 6], and we see that come through on SynthSAEBench as well. Among standard architectures, we see these both performing nearly identically on F1 score at L0=20 and l0=25, and very closely on both MCC and reconstruction. This further shows that SynthSAEBench results reflect what we see in LLM SAEs.

6.3 MP-SAEs overfit superposition noise

We next investigate the effect of superposition noise on SAE performance. We modify the hidden dim of the base SynthSAEBench model, ranging from 256 to 1536, scaling training samples by $(\frac{D}{768})^{(0.6)}$ to account for the change in SAE parameters as recommended by Gao et al. [18]. We train all SAEs at L0=25, as this seems to be a good setting for most SAEs on the base SynthSAEBench model. Results are shown in Figure 6.

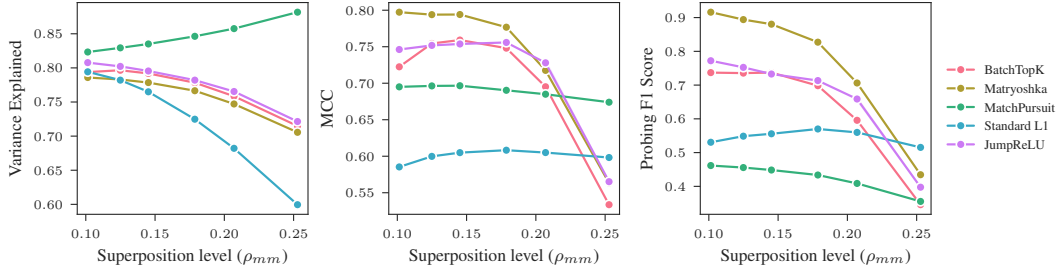


Figure 6: Variance explained (left), MCC (middle), and F1-score (right) for SAEs trained on variants of SynthSAEBench with different levels of superposition. Interestingly, MP-SAEs increase their variance explained at high superposition, implying they are able to overfit on superposition noise.

We see the variance explained decrease with increasing superposition noise except, interestingly, for Matching Pursuit (MP) SAEs. MP-SAEs actually increase variance explained the more superposition noise is present while decreasing MCC and F1-score, implying that MP-SAEs are able to overfit feature overlap due to superposition to improve reconstruction. MP-SAEs use a more expressive encoding process than the simple linear encoder of traditional SAEs, and this extra expressivity can result in overfitting reconstruction without learning the underlying features of the model, perhaps explaining why traditional SAEs with simple linear encoders are so hard to outperform in practice [28]. This insight was only possible because SynthSAEBench provides both ground-truth features (to measure MCC and F1 independently of reconstruction) and controlled superposition levels (to isolate its effect), neither of which is available when benchmarking on LLMs.

7 Related Work

SAE evaluation. Evaluating SAEs on LLMs is challenging due to the lack of ground truth. SAEBench [24] provides a suite of downstream tasks including sparse probing [21], concept erasure

[23], and autointerp [30]. However, these metrics have high variance and measure indirect proxies rather than feature recovery directly. The MCC metric [28] provides a principled way to compare learned features to ground truth. While not directly applicable for SAEs, Interpbench [20] provides a synthetic model for circuit discovery work.

Toy models and synthetic data. Elhage et al. [15] introduced toy models of superposition to study how neural networks represent more features than dimensions. Subsequent work has used small-scale synthetic setups to study specific phenomena: feature absorption in hierarchical features [11], feature hedging under correlation [10], and incorrect L0 behavior [9]. However, these studies use bespoke synthetic models that are not standardized or comparable. Our work provides a unified, large-scale synthetic framework that encompasses superposition, correlation, and hierarchy while providing ground-truth features.

Alternative representation hypotheses. Our work follows the LRH, but other feature hypotheses exist that extend the LRH and are thus natural extensions for our work. Fel et al. [17] introduce the Minkowski Representation Hypothesis, where the softmax operation in the transformer results in polytopes in the representation space. Another extension is feature manifolds [12, 26], where features span manifolds rather than corresponding to a single linear direction.

8 Discussion

SynthSAEBench represents a best-case scenario for SAEs: the Linear Representation Hypothesis holds by construction, with features that are truly linear directions. Yet no SAE architecture we evaluate achieves perfect feature recovery. This is significant because a common response to SAE failures on LLMs is that the representation hypothesis may not hold exactly. Our results show that even when the LRH is perfectly satisfied, current SAE architectures still struggle with superposition, correlation, and hierarchy. The bottleneck is in the SAE architectures and training dynamics themselves, not in the representation hypothesis, strengthening the case for continued architectural innovation.

The SAEs we train are deliberately narrower than the underlying model ($L = 4096 < N = 16,384$), reflecting the LLM regime where SAEs are always narrower than the number of true features in the model [34]. Under MSE, this capacity mismatch incentivizes learned latents to encode mixtures of ground-truth features [10, 9], and this feature mixing or “hedging” is precisely a failure mode current SAEs exhibit. We view imperfect recovery on SynthSAEBench not as a confound but as the target problem: the goal is architectures or training procedures that recover features cleanly even when narrower than the underlying model.

With ground-truth features and controlled ablations, we reproduce key LLM SAE phenomena: Matryoshka SAEs’ high probing performance despite poor reconstruction [7], MP-SAEs’ poor probing despite high reconstruction [8], the gap between SAE probing and supervised probes [22], and the precision-recall trade-off mediated by L0 [11]. We also identify a new failure mode: MP-SAEs exploit superposition noise to improve reconstruction without learning ground-truth features, which may partly explain why traditional SAEs with simple linear encoders remain hard to outperform [28]; more expressive encoding procedures can exploit spurious correlations from superposition.

We emphasize that SynthSAEBench is intended to complement, not replace, LLM benchmarks. Our synthetic model cannot capture all aspects of real neural network representations (see Appendix A for limitations). However, it offers capabilities that LLM benchmarks fundamentally cannot: ground-truth features, controlled ablations, low-noise metrics, and fast iteration. We envision researchers using SynthSAEBench to rapidly prototype and diagnose SAE architectures, then validating promising approaches on LLM benchmarks like SAEBench.

In future work, we hope to extend our framework to Minkowski representations [17] by introducing a “soft” variant of mutual exclusion via softmax, and to feature manifolds, though evaluating SAEs on manifold-structured data is challenging. More broadly, building synthetic models under different representation hypotheses and comparing SAE training dynamics to those observed in LLMs could help test the validity of those hypotheses.

References

- [1] Karl Johan Åström and Richard Murray. *Feedback systems: an introduction for scientists and engineers*. Princeton university press, 2021.
- [2] Kola Ayonrinde. Adaptive sparse allocation with mutual choice & feature choice sparse autoencoders. *arXiv preprint arXiv:2411.02124*, 2024.
- [3] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- [4] Joseph Bloom, Curt Tigges, Anthony Duong, and David Chanin. Saelens. <https://github.com/jbloomAus/SAELens>, 2024.
- [5] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2, 2023.
- [6] Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk sparse autoencoders. *arXiv preprint arXiv:2412.06410*, 2024.
- [7] Bart Bussmann, Noa Nabeshima, Adam Karvonen, and Neel Nanda. Learning multi-level features with matryoshka sparse autoencoders. *arXiv preprint arXiv:2503.17547*, 2025.
- [8] David Chanin. Training matching pursuit saes on llms. LessWrong, 2025. URL <https://www.lesswrong.com/posts/rE43EfHigXcjTmomJ/training-matching-pursuit-saes-on-llms>.
- [9] David Chanin and Adrià Garriga-Alonso. Sparse but wrong: Incorrect l0 leads to incorrect features in sparse autoencoders. *arXiv preprint arXiv:2508.16560*, 2025.
- [10] David Chanin, Tomáš Dulka, and Adrià Garriga-Alonso. Feature hedging: Correlated features break narrow sparse autoencoders. *arXiv preprint arXiv:2505.11756*, 2025.
- [11] David Chanin, James Wilken-Smith, Tomáš Dulka, Hardik Bhatnagar, Satvik Golechha, and Joseph Isaac Bloom. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=R73ybUciQF>.
- [12] Yubei Chen, Dylan Paiton, and Bruno Olshausen. The sparse manifold transform. *Advances in neural information processing systems*, 31, 2018.
- [13] Valérie Costa, Thomas Fel, Ekdeep Singh Lubana, Bahareh Tolooshams, and Demba Ba. From flat to hierarchical: Extracting sparse representations with matching pursuit. *arXiv preprint arXiv:2506.03093*, 2025.
- [14] Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=F76bwRSLek>.
- [15] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.
- [16] Joshua Engels, Eric J Michaud, Isaac Liao, Wes Gurnee, and Max Tegmark. Not all language model features are one-dimensionally linear. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=d63a4AM4hb>.
- [17] Thomas Fel, Binxu Wang, Michael A Lepori, Matthew Kowal, Andrew Lee, Randall Balestriero, Sonia Joseph, Ekdeep S Lubana, Talia Konkle, Demba Ba, et al. Into the rabbit hull: From task-relevant concepts in dino to minkowski geometry. *arXiv preprint arXiv:2510.08638*, 2025.

- [18] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- [19] Rémi Gribonval and Karin Schnass. Dictionary identification—sparse matrix-factorization via ℓ_1 -minimization. *IEEE Transactions on Information Theory*, 56(7):3523–3539, 2010. doi: 10.1109/TIT.2010.2048466.
- [20] Rohan Gupta, Iván Arcuschin Moreno, Thomas Kwa, and Adrià Garriga-Alonso. Interpbench: Semi-synthetic transformers for evaluating mechanistic interpretability techniques. *Advances in Neural Information Processing Systems*, 37:92922–92951, 2024.
- [21] Wes Gurnee, Neel Nanda, Matthew Pauly, Katherine Harvey, Dmitrii Troitskii, and Dimitris Bertsimas. Finding neurons in a haystack: Case studies with sparse probing. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=JYs1R9IMJr>.
- [22] Subhash Kantamneni, Joshua Engels, Senthoran Rajamanoharan, Max Tegmark, and Neel Nanda. Are sparse autoencoders useful? a case study in sparse probing. *arXiv preprint arXiv:2502.16681*, 2025.
- [23] Adam Karvonen, Can Rager, Samuel Marks, and Neel Nanda. Evaluating sparse autoencoders on targeted concept erasure tasks. *arXiv preprint arXiv:2411.18895*, 2024.
- [24] Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. Saebench: A comprehensive benchmark for sparse autoencoders in language model interpretability, 2025. URL <https://arxiv.org/abs/2503.09532>.
- [25] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [26] Eric J Michaud, Liv Gorton, and Tom McGrath. Understanding sparse autoencoder scaling in the presence of feature manifolds. In *Mechanistic Interpretability Workshop at NeurIPS 2025*.
- [27] Chris Olah, Adly Templeton, Trenton Bricken, and Adam Jermyn. April update. <https://transformer-circuits.pub/2024/april-update/index.html>, 2024. URL <https://transformer-circuits.pub/2024/april-update/index.html>.
- [28] Charles O’Neill, Alim Gumran, and David Klindt. Compute optimal inference and provable amortisation gap in sparse autoencoders. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=8forr1FkvC>.
- [29] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=UGpGkLzwpP>.
- [30] Gonçalo Santos Paulo, Alex Troy Mallen, Caden Juang, and Nora Belrose. Automatically interpreting millions of features in large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=EemtbbhJ0Xc>.
- [31] Senthoran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.
- [32] Wilson J Rugh and Jeff S Shamma. Research on gain scheduling. *Automatica*, 36(10):1401–1425, 2000.
- [33] Xiangchen Song, Aashiq Muhamed, Yujia Zheng, Lingjing Kong, Zeyu Tang, Mona T Diab, Virginia Smith, and Kun Zhang. Position: Mechanistic interpretability should prioritize feature consistency in saes. *arXiv preprint arXiv:2505.20254*, 2025.

- [34] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>, May 2024. Accessed on May 21, 2024.
- [35] Yu Wang, Siqi Wu, and Bin Yu. Unique sharp local minimum in l1-minimization complete dictionary learning. *Journal of Machine Learning Research*, 21(63):1–52, 2020.

A Limitations

Synthetic data cannot capture all aspects of real neural network representations. Our generative model assumes linear feature directions, which may not hold for all concepts [16]. The correlation and hierarchy structures, while configurable, are simplified approximations of the complex dependencies in real data. Most importantly, there may be “unknown unknowns”, phenomena in real networks that we have not thought to model. Due to our lack of true ground-truth knowledge of features in LLMs, we do not know how to set hyperparameters like number of features, correlation levels, hierarchy degree, and superposition level. We also do not attempt to model complex feature geometry aside from superposition noise. Synthetic benchmarks should complement, not replace, evaluation on real models.

B Compensating base probabilities for hierarchy

When hierarchy constraints are applied, the effective firing probability of child features is reduced because children can only fire when their parent is active. Similarly, mutual exclusion further reduces effective probabilities since only one sibling can remain active. We implement optional probability compensation to correct for these reductions, ensuring that the *effective* firing rate of each feature matches its specified base probability.

Hierarchy correction. Consider a feature i with base firing probability p_i whose parent has base probability p_{parent} . Without compensation, the effective probability of feature i firing is approximately $p_i \cdot p_{\text{parent}}$, since the child can only fire when the parent fires. To compensate, we scale up the sampling probability by a correction factor:

$$\gamma_i^{\text{hier}} = \frac{1}{p_{\text{parent}}}. \quad (19)$$

After sampling with corrected probability $\tilde{p}_i = \min(1, p_i \cdot \gamma_i^{\text{hier}})$ and applying hierarchy constraints, the effective firing rate is approximately p_i . The clamp at 1 is active only when $p_i > p_{\text{parent}}$; in our setup this never occurs, because features are assigned to tree nodes in breadth-first order while base probabilities follow a non-increasing Zipfian distribution, so a child always has $p_i \leq p_{\text{parent}}$.

For deeper hierarchies, this correction is applied recursively. A feature at depth d with ancestors having probabilities $p_1, p_2, \dots, p_{d-1}$ would naively have effective probability $p_i \cdot \prod_{k=1}^{d-1} p_k$. However, since each ancestor also receives its own correction, the child only needs to correct for its immediate parent’s base probability.

Mutual exclusion correction. When a parent node has mutually exclusive children, at most one child can remain active per sample. This introduces additional probability reduction beyond hierarchy constraints.

Consider an ME group with children having base probabilities $p_1, p_2, \dots, p_m$ under a parent with probability p_P . After hierarchy correction, conditioned on the parent firing, child j fires with probability approximately p_j/p_P . When multiple children fire, one is randomly selected as the winner.

For child i , the expected number of competing siblings (given parent fires) is:

$$\mathbb{E}[\text{competitors}] = \sum_{j \neq i} \frac{p_j}{p_P}. \quad (20)$$

453 To first order, the probability of child i being deactivated by ME is proportional to this expected
 454 competitor count. We apply a multiplicative ME correction:

$$\gamma_i^{\text{ME}} = 1 + \sum_{j \neq i} \frac{p_j}{p_P}. \quad (21)$$

455 The total correction factor for a feature in an ME group is $\gamma_i = \gamma_i^{\text{hier}} \cdot \gamma_i^{\text{ME}}$.

456 **Limitations.** This compensation is approximate and assumes independence between sibling firings.
 457 In practice, correlations between features (introduced via the correlation matrix Σ) can cause devia-
 458 tions from the target probabilities. The correction also does not account for higher-order effects when
 459 multiple levels of hierarchy and ME interact. Nevertheless, we find empirically that compensation
 460 substantially improves the match between specified and effective firing probabilities.

461 C Parent-scaled child magnitudes

462 By default, hierarchy constraints apply binary gating: a child feature retains its sampled magnitude
 463 when its parent is active, and is zeroed out otherwise. We additionally support *parent-scaled*
 464 *magnitudes*, where the child’s activation is modulated by the parent’s normalized activation strength.
 465 Formally, for a child feature with coefficient c_{child} whose parent has coefficient c_{parent} and mean firing
 466 magnitude $\bar{\mu}_{\text{parent}}$:

$$c_{\text{child}} \leftarrow c_{\text{child}} \cdot \frac{c_{\text{parent}}}{\bar{\mu}_{\text{parent}}}, \quad (22)$$

467 where $\bar{\mu}_{\text{parent}}$ is the precomputed mean magnitude of the parent feature. When the parent is inactive
 468 ($c_{\text{parent}} = 0$), the child is zeroed out as in the standard case. Dividing by $\bar{\mu}_{\text{parent}}$ normalizes the scaling
 469 so that the child’s expected magnitude is preserved on average.

470 This models the intuition that more specific concepts should be modulated by the intensity of their
 471 parent concept. For instance, features associated with specific dog breeds should fire more strongly
 472 when the “dog” feature is strongly active. Without this scaling, the child magnitude is independent of
 473 how strongly the parent fires, which may be unrealistic for many natural concepts.

474 Parent-scaled magnitudes can be enabled independently per parent node, and compose with both
 475 mutual exclusion and the probability compensation described in Appendix B. The scaling is applied
 476 after binary gating and mutual exclusion resolution, so the cascading order remains: (1) parent gating,
 477 (2) mutual exclusion, (3) magnitude scaling.

478 D Low-rank correlation matrix generation

479 For scalability with large numbers of features, we use a low-rank representation of the correlation
 480 matrix rather than storing the full $N \times N$ matrix. This reduces storage from $O(N^2)$ to $O(Nr)$ where
 481 r is the rank.

482 The correlation structure is represented as:

$$\Sigma = \mathbf{F}\mathbf{F}^\top + \text{diag}(\boldsymbol{\delta}) \quad (23)$$

483 where $\mathbf{F} \in \mathbb{R}^{N \times r}$ is a factor matrix and $\boldsymbol{\delta} \in \mathbb{R}^N$ contains diagonal residual variances.

484 **Generation procedure.** Given rank r and correlation scale s , we generate the factor matrix by
 485 sampling from a scaled normal distribution:

$$F_{ij} \sim s \cdot \mathcal{N}(0, 1) \quad (24)$$

486 The diagonal term is then computed to ensure unit diagonal in the implied correlation matrix:

$$\delta_i = 1 - \sum_{j=1}^r F_{ij}^2 \quad (25)$$

487 **Numerical stability.** If any diagonal term δ_i falls below a minimum threshold (we use 0.01), the
 488 entire factor matrix is scaled down to ensure all diagonal terms remain valid. Specifically, we compute
 489 a scale factor:

$$\text{scale} = \sqrt{\frac{1 - \delta_{\min}}{\max_i \sum_j F_{ij}^2}} \quad (26)$$

490 and apply $\mathbf{F} \leftarrow \text{scale} \cdot \mathbf{F}$, then recompute δ .

491 **Efficient sampling.** Sampling from this low-rank structure requires only $O(Nr)$ computation per
 492 batch:

$$\mathbf{g} = \mathbf{F}\boldsymbol{\epsilon} + \sqrt{\delta} \odot \boldsymbol{\eta}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_r), \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N) \quad (27)$$

493 where $\odot$ denotes elementwise multiplication. The resulting $\mathbf{g}$ has the desired correlation structure
 494 $\mathbb{E}[\mathbf{g}\mathbf{g}^\top] = \boldsymbol{\Sigma}$.

495 E Configurable firing distributions

496 Both per-feature firing probabilities p_i and per-feature magnitude parameters (μ_i, σ_i) admit several
 497 configurable distributions in our framework.

498 **Firing probabilities.** The distribution of p_i across features significantly impacts SAE training dy-
 499 namics: very rare features are difficult to learn, while very frequent ones dominate reconstruction loss.
 500 We use a Zipfian distribution as the default, motivated by the observation that concept frequencies in
 501 natural data follow power laws [2, 16]: $p_i \propto i^{-\alpha}$, scaled to lie in $[p_{\min}, p_{\max}]$. We also support linear
 502 decay (p_i interpolates linearly from $p_{\max}$ to $p_{\min}$), uniform random ($p_i \sim \text{Uniform}(p_{\min}, p_{\max})$),
 503 and constant ($p_i = p$). Arbitrary distributions can be plugged in.

504 **Firing magnitudes.** When a feature fires, its coefficient is sampled from a rectified Gaussian:
 505 $c_i = \text{ReLU}(\mu_i + \sigma_i \epsilon_i)$, $\epsilon_i \sim \mathcal{N}(0, 1)$. The per-feature mean μ_i and standard deviation σ_i can
 506 each take constant, linear-interpolated, exponential-interpolated, or folded-normal forms (e.g., $\sigma_i \sim$
 507 $|\mathcal{N}(\mu_\sigma, \sigma_\sigma^2)|$); the folded normal creates heterogeneous magnitude variability. Arbitrary distributions
 508 over μ_i and σ_i are supported.

509 F SAE training procedures

510 We train all SAEs using the Adam optimizer [25] with learning rate 3×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.999$,
 511 and batch size 1024. The learning rate decays linearly to zero over the final third of training. All
 512 SAEs have width 4096 and are trained on 200M samples from SynthSAEBench. Below we describe
 513 architecture-specific training details.

514 F.1 BatchTopK SAEs

515 BatchTopK SAEs [6] use a soft top- k selection that allows the number of active features to vary per
 516 sample while maintaining a target average L0 across the batch. The target L0 is set directly via the k
 517 parameter, so no autotuning is required.

518 To prevent dead latents, we use the TopK auxiliary loss from Gao et al. [18]. This auxiliary loss has
 519 dead latents reconstruct the residual error from live latents, providing gradient signal to features that
 520 would otherwise receive none. Following the heuristic from Gao et al. [18], we set $k_{\text{aux}} = D/2$ where
 521 D is the input dimension, and scale the loss by $\min(\text{num_dead}/k_{\text{aux}}, 1)$ to reduce its magnitude when
 522 few latents are dead.

523 F.2 JumpReLU SAEs

524 Our JumpReLU training procedure follows the original procedure in Rajamanoharan et al. [31]
 525 matching the procedure used by SAE Bench, except we tune some hyperparameters to perform better
 526 in the synthetic setting. We use initial JumpReLU threshold of 1.0, bandwidth of 1.0, and use a latent
 527 norm of 0.5 at initialization. We find that with these settings, JumpReLU SAEs perform well on
 528 SynthSAEBench and we do not see many dead latents.

For sparsity control, we use an initial penalty of 1.0 with no warm-up and immediately adjust using an L0 coefficient autotuner (Appendix G) to hit a target L0.

F.3 Standard L1 SAEs

For standard L1 SAEs, we follow the procedure from Olah et al. [27], training with a weighted combination of reconstruction MSE and an L1 penalty on feature activations. The L1 coefficient is warmed up linearly over the first third of training, after which we use the L1 coefficient autotuner (Appendix G) to achieve target L0 values. No auxiliary loss is used.

F.4 Matryoshka SAEs

We train Matryoshka SAEs [7] using the BatchTopK activation function with nested prefixes. We use prefix sizes $\mathcal{M} = \{128, 512, 2048, 4096\}$. Each prefix is trained to reconstruct the input independently.

While the original Matryoshka SAEs work uses the standard TopK SAE auxiliary loss described in Appendix F.1, we find that using a Matryoshka-optimized auxiliary loss results in better performance, especially at low L0.

Matryoshka auxiliary loss. The standard TopK auxiliary loss [18] trains dead latents to reconstruct the residual error of the full SAE. However, in a Matryoshka SAE, dead latents at early prefixes (e.g., the first 128 latents) face a very different reconstruction residual than dead latents at later prefixes. Training all dead latents against the full SAE’s residual provides a poor learning signal for latents in smaller prefixes, since those latents need to help reconstruct a much larger residual.

Our Matryoshka auxiliary loss instead computes a separate auxiliary loss for each matryoshka prefix. For prefix $m \in \mathcal{M}$, let $\hat{a}_m$ be the reconstruction using the first m latents, and let the dead latents within the range $[m_{\text{prev}}, m)$ be denoted $\mathcal{D}_m$. The auxiliary loss for prefix m is:

$$\mathcal{L}_{\text{aux},m} = s_m \cdot \|W_{\text{dec},\mathcal{D}_m} f_{\text{aux},m} - (a - \hat{a}_m)_{\text{detach}}\|_2^2, \quad (28)$$

where $f_{\text{aux},m}$ are the top- $\min(k_{\text{aux}}, |\mathcal{D}_m|)$ activations among dead latents in $[m_{\text{prev}}, m)$, using only the corresponding portion of the encoder pre-activations. The scale factor $s_m = \min(|\mathcal{D}_m|/k_{\text{aux}}, 1)$ reduces the loss magnitude when few latents in that prefix are dead. The residual $(a - \hat{a}_m)$ is detached to prevent the auxiliary loss from affecting the main reconstruction pathway.

This per-prefix formulation ensures that dead latents in early prefixes receive gradient signal appropriate to their level’s reconstruction error, rather than the much smaller residual of the full SAE.

We explore dead latents further in Appendix H.

G Autotuning sparsity coefficients

For SAE architectures that use a sparsity-inducing loss (standard L1 SAEs and JumpReLU SAEs), the sparsity coefficient λ controls the trade-off between reconstruction and sparsity. However, the relationship between λ and the resulting L0 sparsity is nonlinear and model-dependent, making it difficult to train SAEs at a specific target L0 for fair comparison. We implement an autotuning controller that dynamically adjusts a multiplier m on the sparsity coefficient during training to achieve a target L0.

Controller design. We use a rate-dampened integral controller [1] that adjusts the effective sparsity coefficient $\lambda_{\text{eff}} = \lambda \cdot m$ based on the deviation from target L0. The controller maintains:

- A smoothed L0 estimate $\bar{\ell}_t$ using exponential moving average (EMA) with smoothing factor α
- A smoothed rate of L0 change $\dot{\ell}_t$ (the derivative estimate)

At each training step, given batch L0 measurement ℓ_t :

$$\bar{\ell}_t = \alpha \bar{\ell}_{t-1} + (1 - \alpha) \ell_t \quad (29)$$

$$\dot{\ell}_t = \alpha_r \dot{\ell}_{t-1} + (1 - \alpha_r) (\bar{\ell}_t - \bar{\ell}_{t-1}) \quad (30)$$

571 where $\alpha = 0.99$ and $\alpha_r = 0.95$ are the smoothing factors for position and rate respectively.

572 **Gain scheduling.** Gain scheduling [32] adapts controller parameters based on operating conditions.
 573 The key insight is that when the system is converging toward the target (error decreasing), we should
 574 reduce the controller gain to prevent overshoot. We detect convergence when the error and rate have
 575 opposite signs:

$$\text{converging} = (\bar{\ell}_t - \ell^*) \cdot \dot{\ell}_t < 0 \quad (31)$$

576 where ℓ^* is the target L0. When converging, we reduce the gain by a factor $\gamma_c = 0.01$.

577 **Bounded adjustment.** To ensure stable behavior, we use a tanh nonlinearity to bound the adjustment
 578 magnitude:

$$\Delta = K_i \cdot g \cdot \tanh \left(\left| \frac{\bar{\ell}_t - \ell^*}{\ell^*} \right| \cdot s \right) \quad (32)$$

579 where $K_i = 3 \times 10^{-4}$ is the integral gain, $g \in \{\gamma_c, 1\}$ is the scheduled gain, and $s = 10$ is the gain
 580 scale. The multiplier is then updated multiplicatively:

$$m_{t+1} = \begin{cases} m_t \cdot (1 + \Delta) & \text{if } \bar{\ell}_t > \ell^* \\ m_t \cdot (1 - \Delta) & \text{if } \bar{\ell}_t < \ell^* \end{cases} \quad (33)$$

581 The multiplier is clamped to $[0.01, 100]$ to prevent extreme values.

582 **Integration with SAE training.** For standard L1 SAEs, the autotuner modulates the L1 coefficient:
 583 $\lambda_1^{\text{eff}} = \lambda_1 \cdot m$. For JumpReLU SAEs, it modulates the L0 penalty coefficient similarly. The autotuner
 584 state is updated after each training batch, and the new multiplier is applied to the next batch’s loss
 585 computation.

586 This approach allows us to train SAEs at precisely matched L0 values across different architectures,
 587 enabling fair comparison of their reconstruction and feature recovery quality.

588 H Dead latents in SynthSAEBench

589 Dead latents are a problem in SAE training in general, with SynthSAEBench being no exception. We
 590 evaluate dead latents over 1M samples for the experiments in this section.

591 H.1 Dead latents in BatchTopK and Matryoshka SAEs

592 BatchTopK and especially Matryoshka BatchTopK SAEs seem to struggle with dead latents at low
 593 L0 in SynthSAEBench, even with the aux loss reviving dead latents. We can see this in Figure 7,
 594 where dead latents increase at L0=15 for BatchTopK and especially Matryoshka SAEs. We find that
 595 these latents are actually not entirely “dead”, as they will still fire occasionally, but far less frequently
 596 than our dead latent window. This suggests that SAEs are settling into poor local minima.

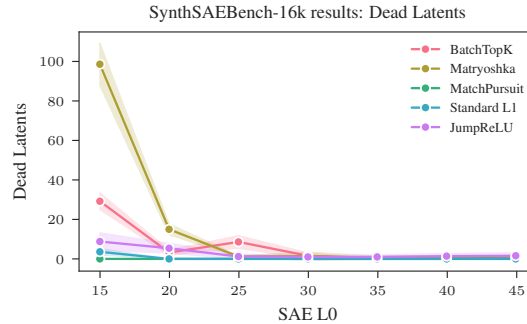


Figure 7: Dead latents vs L0 for SynthSAEBench SAEs. Shaded area is 1 stdev with 5 random seeds.

597 We also find that using the Matryoshka-optimized variant of the auxiliary loss described in Ap-
 598 pendix F.4 results in a better Matryoshka SAE with fewer dead latents, especially at low L0. We
 599 show SAE quality results comparing these auxiliary losses for Matryoshka SAEs in Figure 8, and
 600 show dead latents in Figure 9.

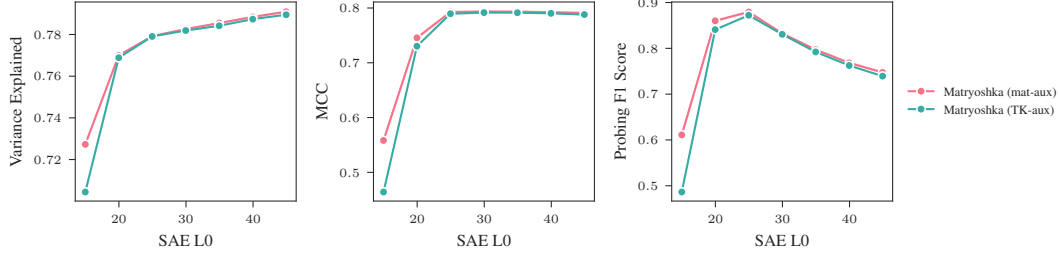


Figure 8: Comparing Matryoshka SAEs trained with standard TopK auxiliary loss (TK-aux) and a Matryoshka-optimized TopK auxiliary loss (mat-aux). The matryoshka-optimized loss results in better SAEs, but the difference is especially pronounced at low L0.

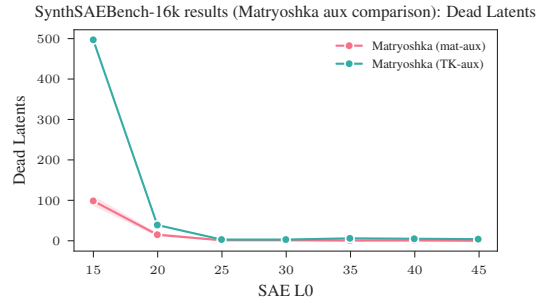


Figure 9: Dead latents for Matryoshka SAEs trained with standard TopK auxiliary loss (TK-aux) and a Matryoshka-optimized TopK auxiliary loss (mat-aux). The matryoshka-optimized loss results in fewer dead latents, especially pronounced at low L0.

H.2 Dead latents in Matching Pursuit and Standard L1 SAEs.

Matching Pursuit (MP) SAEs seem to never have any dead latents in SynthSAEBench, no matter the setting. This is impressive and a clear benefit of this architecture. Standard L1 SAEs also did not have any problems with dead latents in our experiments as long as the L1 coefficient is linearly warmed up as suggested by Olah et al. [27].

I SynthSAEBench sample generation performance

We next investigate the performance characteristics of the synthetic data generation process as a function of number of features in the synthetic model. We use our base SynthSAEBench model, but vary the number of features in the model from 2^7 (128) to 2^{20} (1M). We keep the same 3-level mutually-exclusive hierarchy scaled relative to the size of the data model. We then sample 100 batches of size 1024 on an Nvidia H100 GPU and benchmark the sample throughput of the model. Results are shown in Figure 10.

Feature generation ranges from 600K samples / sec (2.5 min for 100M samples) for models with under 1000 features to 7K samples / sec (4hr for 100M samples) for a model with 1M features. We decided on 16k as a good compromise between these two extremes. At 16k features, the model generates at 300K samples / sec, or 5 min for 100M samples.

J Noise in SAE Bench metrics

One motivation for this work is that existing SAE benchmarks like SAE Bench, while extremely important and indispensable for SAE architecture development, tend to be noisy. For instance, we show SAE Bench Spurious Correlation Removal (SCR), Targeted Probe Perturbation (TPP), and Sparse Probing metrics from the SAE Bench paper for Gemma-2-2b layer 12 width 16k SAEs in Figure 11.

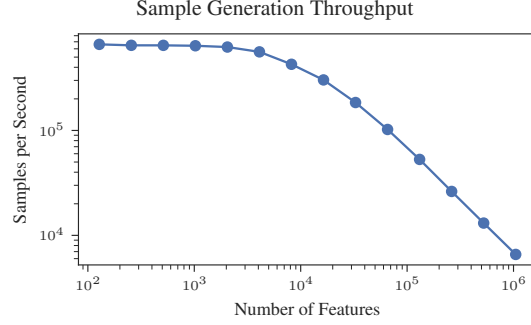


Figure 10: Synthetic model throughput by number of features.

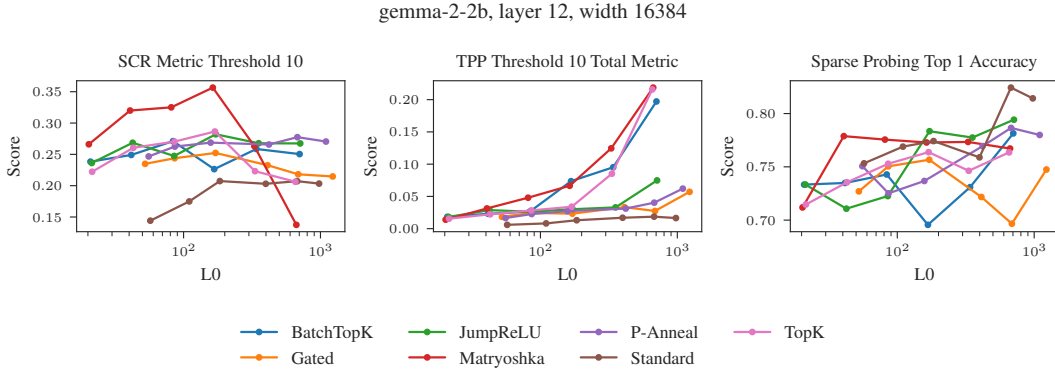


Figure 11: SAE Bench SCR, TPP, and Sparse Probing metrics for Gemma-2-2b layer 12 width 16k SAEs.

While there are clear trends in some metrics, overall there is still a lot of random noise that makes it difficult to make fine-grained judgements about SAE architecture differences. Cutting through this noise requires running many seeds of SAEs, which is often infeasibly expensive for LLM SAEs.

K Exploring Superposition

How much pairwise feature interference does a given hidden dimension impose, and how does it scale with the number of features? Below we give a closed-form characterisation of ρ_{mm} for random unit-norm directions, which is essentially indistinguishable from the optimum in the regimes relevant to neural networks.

Distribution of pairwise cosine similarity. For two independent unit-norm vectors $u, v \in \mathbb{R}^D$ drawn uniformly from the sphere, the squared cosine similarity $\langle u, v \rangle^2$ follows a $\text{Beta}(1/2, (D-1)/2)$ distribution. Letting F_D denote its CDF, the maximum absolute cosine similarity of one direction against the other $N - 1$ directions has CDF

$$P\left(\max_{j \neq i} |\langle u_i, u_j \rangle| \leq x\right) = [F_D(x^2)]^{N-1}, \quad (34)$$

and $\mathbb{E}[\rho_{\text{mm}}]$ follows by direct integration, with no Monte Carlo simulation required. Spherical packing bounds further show that, in the regime $N \ll 2^D$ that covers all practical neural network widths, optimally-placed directions achieve essentially the same ρ_{mm} as random ones, so the analysis applies whether or not features are explicitly orthogonalised.

Asymptotic scaling. Taylor expanding Eq. (34) around $x = 0$ yields the simple approximation

$$\rho_{\text{mm}} \approx \sqrt{\frac{\ln N}{D}}. \quad (35)$$

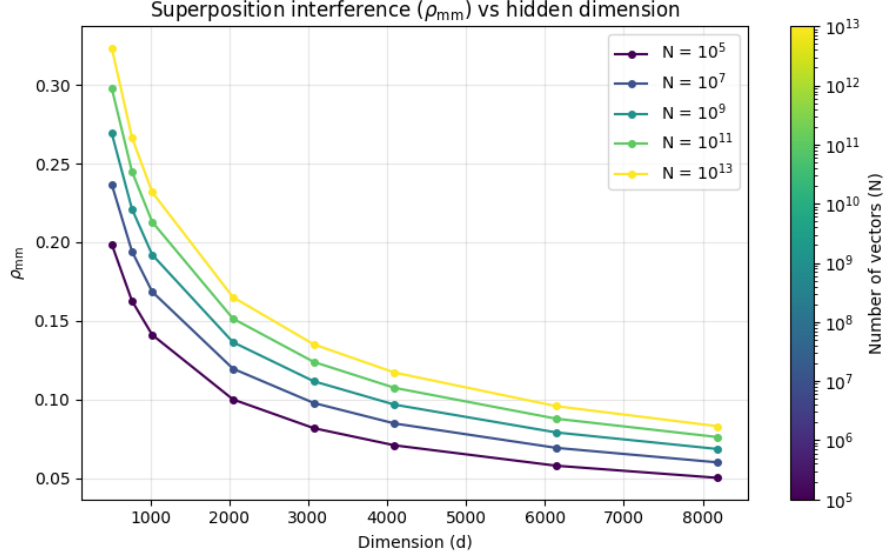


Figure 12: Mean max absolute cosine similarity ρ_{mm} as a function of hidden dimension D , computed analytically from Eq. (34) for random unit-norm feature directions, with the number of features N ranging from 10^5 to 10^{13} .

Two consequences follow. First, interference shrinks as $O(1/\sqrt{D})$ in the hidden dimension. Second, the dependence on the number of features is only logarithmic: scaling N by $100\times$ adds only $\sqrt{\ln 100} \approx 2.1$ to the numerator, an effect easily absorbed by a modest increase in D . Hidden dimension dominates feature count in determining how much superposition is feasible.

Implications for SynthSAEBench. Figure 12 plots ρ_{mm} from Eq. (34) against D for N ranging from 10^5 to 10^{13} . Even at $N = 10^{13}$, ρ_{mm} falls below 0.1 once $D \geq 6000$. Conversely, SynthSAEBench ($N = 16,384$, $D = 768$) yields $\rho_{mm} \approx 0.15$, which is substantially more pairwise interference than would arise for trillions of features at a modern LLM hidden dimension. This is roughly equivalent to the amount of superposition we would expect in 1 Billion features in 2k hidden dim, roughly the size of Gemma-2-2b, which we feel is a reasonable baseline.

L Logistic regression probes on SynthSAEBench

In Section 5.1, we note that the best SAE architecture (Matryoshka BatchTopK) achieves a peak probing F1 of approximately 0.88, consistent with LLM SAE findings that SAEs underperform supervised probes [22]. To quantify this gap on SynthSAEBench, we train logistic regression probes directly on hidden activations to classify ground-truth feature firings.

Setup. We sample 2M activations from SynthSAEBench and evaluate the first 4,096 features, which are the highest-frequency features due to the Zipfian ordering of firing probabilities. We train one linear probe per feature simultaneously using a batched logistic regression model (a single weight matrix $W \in \mathbb{R}^{F \times D}$ and bias $b \in \mathbb{R}^F$, where F is the number of probed features). Training uses Adam with learning rate 3×10^{-3} , cosine annealing over 10,000 steps, batch size 4,096, L_2 weight decay of 10^{-3} , and class-imbalance-corrected binary cross-entropy loss. We use an 80/20 train/test split, and tune a per-feature classification threshold on a 200K-sample validation subset drawn from the training set by sweeping 499 thresholds and selecting the one that maximizes F1 per feature.

Results. Results are shown in Table 1. The probes achieve a mean F1 of 0.974 and mean AUC of 0.9999, substantially outperforming the best SAE probing F1 of ≈ 0.88 . This confirms that the gap between SAE probing and supervised probing observed in LLMs [22] is reproduced in SynthSAEBench, and that this gap is not an artifact of LLM evaluation noise but reflects a genuine limitation of current SAE architectures.

Table 1: Logistic regression probe results on SynthSAEBench (first 4,096 highest-frequency features).

Metric	Mean	Median
AUC	0.999868	0.999982
Accuracy	0.999837	0.999873
Precision	0.979978	0.985310
Recall	0.967982	0.971107
F1	0.973877	0.977504

M Extended results

We include extended results for the experiments in the paper.

M.1 SynthSAEBench extended results

Full results for SynthSAEBench are shown in Figure 13.

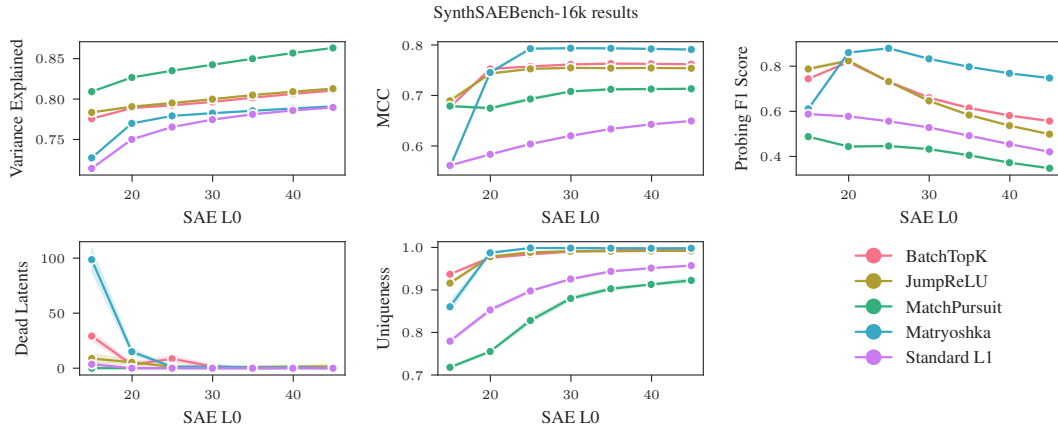


Figure 13: Full results for SynthSAEBench. Shaded area represents 1 stdev with 5 seeds per SAE.

M.2 Ablation: feature correlation strength

Next, we explore the effect of varying the strength of the random correlations between features. We vary the correlation scale used to generate the random correlation matrix from 0 to 0.25, and train SAEs with L0=25. Results are shown in Figure 14.

The effect of varying correlation strength are less dramatic than varying superposition, but still interesting. We see that variance explained increases for all SAEs as correlation increases except for Matryoshka SAEs, but this appears to be due to dead latents rather than a fundamental architectural issue. This is consistent with previous work showing that SAEs are able to exploit feature correlation to increase reconstruction by mixing correlated features [10, 9]. We see slight decreases in probing F1 score as correlation increases, but mixed results on MCC that are harder to judge.

Likely the effect of feature correlation in SynthSAEBench is overshadowed by the effect of superposition noise and hierarchy (itself an even more extreme form of correlation).

M.3 Ablation: feature correlation rank

Next, we explore the effect of varying the rank of the low-rank correlations between features. We vary the rank from 1 to 100, and train SAEs with L0=25. We use correlation strength 0.075 throughout. Results are shown in Figure 15.

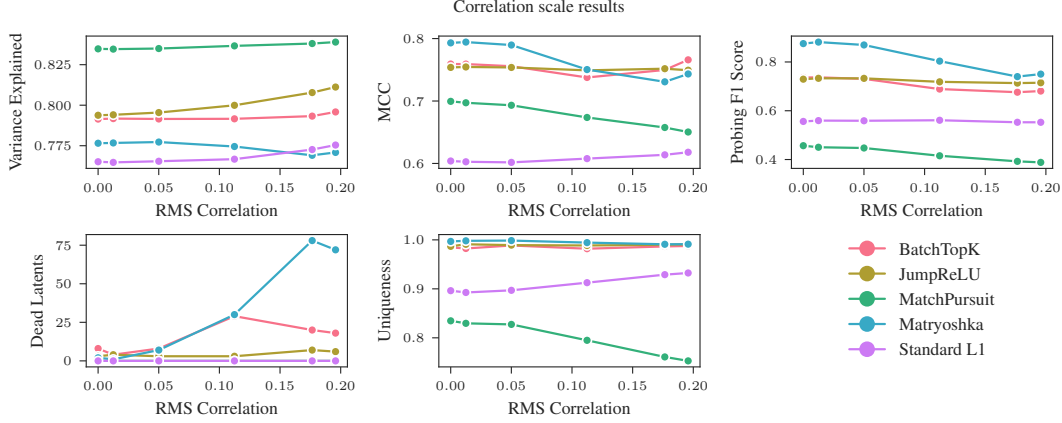


Figure 14: Results varying correlation strength, while keeping the remaining model hyperparameters set at default values for SyntheSAEBench-16k.

688 The effect of varying correlation rank is effectively negligible. This is not surprising, since the
689 correlation rank just changes the types of correlation patterns that are possible, but does not change
690 the underlying strength or quantity of correlations.

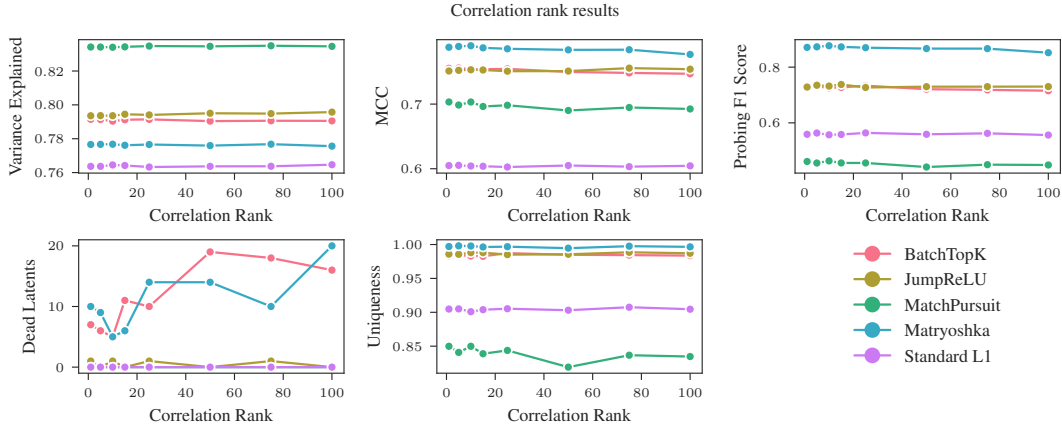


Figure 15: Results varying correlation rank, with correlation strength 0.075, and keeping the remaining model hyperparameters set at default values for SyntheSAEBench-16k.

691 M.4 Ablation: feature firing probability

692 We perform an ablation study where we decrease the base firing probabilities of all features by a
693 multiplier, effectively decreasing the L0 of the model. We evaluate each SAE at $L0 = 25 * p$ where p
694 is the firing multiplier. Results are shown in Figure 16. We see broadly the same pattern as in the
695 main experiments, with Matryoshka being best at quality metrics, and Matching Pursuit having the
696 best reconstruction. Interestingly, Matching Pursuit SAEs increase variance explained with higher L0
697 while other SAEs do not.

698 M.5 Ablation: feature firing stdev

699 We perform an ablation study where we set all features to have the same firing magnitude stdev, and
700 then vary the stdev from 0.5 to 10.0. Each SAE is trained at $L0=25$. Results are shown in Figure 17.
701 At high stdev, F1 score and variance explained drops for every SAE. This makes sense since it is
702 hard to distinguish a low-magnitude firing from superposition noise when there is such a wide range
703 of potential firing magnitudes for each feature. Interestingly, however, standard L1 SAEs achieve
704 the best MCC of all SAEs at high firing magnitude stdev. We are not sure why this is, but it could

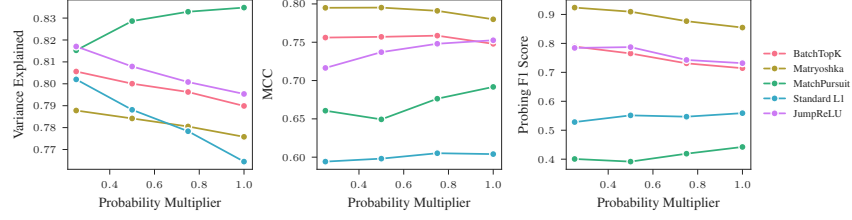


Figure 16: Ablation study varying the base firing probabilities of the SynthSAEBench model. We scale the L0 of the SAEs to match the scaled firing probabilities based on the default $L_0=25$ ($L_0=12.5$ when probability multiplier is 0.5). We see broadly the same pattern as in the main experiments, with Matryoshka being best at quality metrics, and Matching Pursuit having the best reconstruction. Interestingly, Matching Pursuit SAEs increase variance explained with higher L0 while other SAEs do not.

be worth studying further to try to understand why a simple L1 penalty does so well in this regime compared to the other SAE architectures.

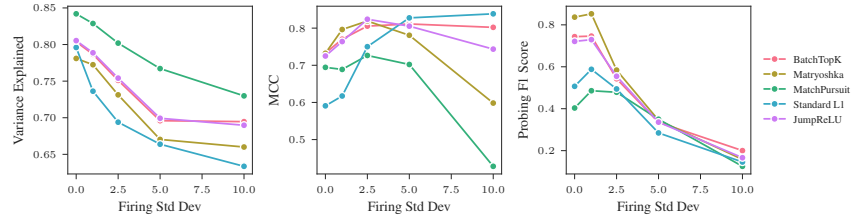


Figure 17: Ablation study varying the feature firing magnitude stddev of the SynthSAEBench model. We see broadly the same pattern as in the main experiments, with Matryoshka being best at quality metrics, and Matching Pursuit having the best reconstruction. Interestingly, we see a dichotomy between MCC and F1 score at high firing stddev, where high stddev improves MCC but reduces F1 score for all SAEs. This makes sense as high firing magnitude variance makes feature directions more prominent for the SAE, but means there are more low-magnitude samples that are difficult for the SAE to disambiguate from superposition noise.

M.6 Ablation: deeper hierarchy

We perform an ablation study using a much deeper hierarchy (128 root nodes, 2 children per node, max depth of 6). All SAEs are trained at $L_0=25$. Results are shown in Figure 18. We see the same general pattern as in SynthSAEBench, where the Matryoshka SAE is best by quality metrics (MCC and F1), despite poor reconstruction (variance explained). The MP-SAE achieves best reconstruction, but scores poorly on quality metrics. JumpReLU and BatchTopK are in-between on both SAE quality and reconstruction, and the Standard SAE performs poorly on all metrics.

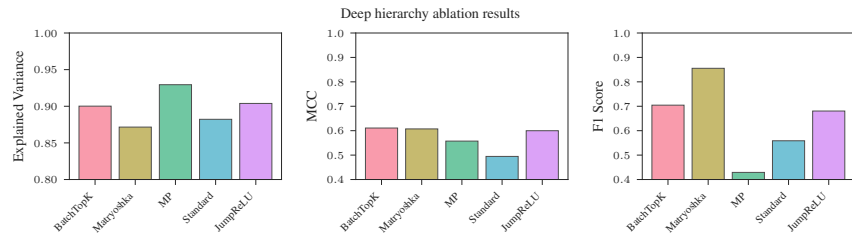


Figure 18: Ablation study using a much deeper hierarchy (128 root nodes, 2 children per node, max depth of 6). All SAEs are trained at $L_0=25$. We see the same general pattern as in SynthSAEBench.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 state our contributions: a large-scale synthetic SAE benchmark with ground-truth features and firings, reproduction of several known LLM SAE phenomena, and identification of a new failure mode in which Matching Pursuit SAEs overfit superposition noise. Sections 5–6 substantiate each claim empirically.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Discussed in Appendix A: features are strictly linear by construction, correlation and hierarchy structures are simplified approximations, we lack ground truth for choosing benchmark hyperparameters that match real LLMs, and there may be unmodeled phenomena in real networks.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not contain formal theorems requiring proofs. The mathematical derivations it does include (closed-form analysis of pairwise cosine similarity in Appendix K and probability compensation factors in Appendix B) state their assumptions inline.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The benchmark configuration is fully specified in Section 5; SAE training procedures and architecture-specific settings in Appendix F; the sparsity autotuning controller in Appendix G; and the logistic-regression probe setup in Appendix L. Code is provided as part of the submission.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code is provided as part of the submission. The pretrained synthetic benchmark model and experiment scripts will be released at a public URL at camera-ready time; URLs are omitted from the submission to preserve double-blind anonymity.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5 specifies benchmark hyperparameters and SAE width/training-sample recommendations. Appendix F lists SAE training settings (Adam, learning rate, batch size, sample count) and architecture-specific details. Appendix G describes the sparsity autotuning controller. Appendix L details probe training, including thresholds and validation splits.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The main results in Section 6 are computed over 5 random seeds per L0 setting; shaded regions in figures denote one standard deviation across seeds, as stated in the figure captions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 5 reports SAE training compute (15–20 minutes per SAE on a single NVIDIA H100 GPU at batch size 1024 with 200M training samples). Appendix I reports synthetic data generation throughput across model sizes on the same hardware.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics. The work is methodological, involves no human subjects and no scraped or sensitive data, and we do not foresee misuse concerns specific to a synthetic SAE benchmark.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: Improved SAE evaluation tools support mechanistic interpretability research, which contributes to understanding and debugging LLMs, and indirectly to safer deployment. We do not foresee direct negative societal applications specific to a synthetic benchmark for sparse autoencoders.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are a synthetic-data generation toolkit and a pretrained synthetic activation model used to evaluate SAE architectures. They contain no real data, no model trained on text or images, and pose no foreseeable misuse risk.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing assets are cited where used: SAELens [4] (MIT-licensed; we extend it as part of this work), Pythia-160m [3] (used only as a reference for matching activation statistics), and the SAEBench evaluation suite [24]. We use these within their license terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new assets (the SynthSAEBench toolkit and pretrained synthetic activation model) are documented in Section 5 and the data-generation appendices (Appendices B–E), in the SAELens project documentation that we extend, and in the supplementary material accompanying the submission.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The research involves no crowdsourcing and no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

1032 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
 1033 or other labor should be paid at least the minimum wage in the country of the data
 1034 collector.

1035 **15. Institutional review board (IRB) approvals or equivalent for research with human**
 1036 **subjects**

1037 Question: Does the paper describe potential risks incurred by study participants, whether
 1038 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
 1039 approvals (or an equivalent approval/review based on the requirements of your country or
 1040 institution) were obtained?

1041 Answer: [N/A]

1042 Justification: The research involves no human subjects, so IRB approval is not applicable.

1043 Guidelines:

1044 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
 1045 with human subjects.

1046 • Depending on the country in which research is conducted, IRB approval (or equivalent)
 1047 may be required for any human subjects research. If you obtained IRB approval, you
 1048 should clearly state this in the paper.

1049 • We recognize that the procedures for this may vary significantly between institutions
 1050 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
 1051 guidelines for their institution.

1052 • For initial submissions, do not include any information that would break anonymity (if
 1053 applicable), such as the institution conducting the review.

1054 **16. Declaration of LLM usage**

1055 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 1056 non-standard component of the core methods in this research? Note that if the LLM is used
 1057 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
 1058 scientific rigor, or originality of the research, declaration is not required.

1059 Answer: [N/A]

1060 Justification: LLMs are not part of the core methodology. The synthetic benchmark and
 1061 SAE evaluation framework do not use LLMs as components; LLMs were used only for
 1062 writing assistance.

1063 Guidelines:

1064 • The answer [N/A] means that the core method development in this research does not
 1065 involve LLMs as any important, original, or non-standard components.

1066 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
 1067 be described.